



GAME THEORY

Smart Traffic Control

Multi-Agent Strategic Interaction

Student Name

ID

Haidy Mohamed

2305309

Arwa Ayman

2305137

Rodina Mohamed

2305287

Shaimaa Mahmoud

2305265

Salma Hassanien

2305509



Table of Contents

1. Introduction & Project Overview 2. System

Architecture

3. Game Theory Concepts Applied

3.1 Normal Form Game & Payoff Matrix

3.2 Iterated Elimination of Strictly Dominated Strategies (IESDS) 3.3 Pure Nash Equilibrium

3.4 Mixed Nash Equilibrium

3.5 Best Response Analysis

3.6 Beliefs & Bayesian Updating

3.7 Extensive Form & Backward Induction

4. Intelligent Agent Types

5. Simulation Design &

Environment 6. Results &

Performance Analysis 7.

Conclusions

1. Introduction & Project Overview

This project presents a comprehensive multi-agent intelligent system for Smart Traffic Control, developed as part of the Game Theory Applications in Intelligent Systems course. The central objective is to model the strategic interactions between autonomous vehicles or traffic agents at a controlled intersection, using formal game-theoretic frameworks to derive optimal decision strategies.

Real-world traffic management is inherently a strategic environment: each vehicle's best action depends on what other vehicles decide to do. By framing this as a game between rational (or learning) agents, we can apply the rich toolkit of game theory — normal-form games, Nash equilibria, backward induction, mixed strategies, and belief updating — to design agents that behave intelligently and predictably.

1.1 Motivation

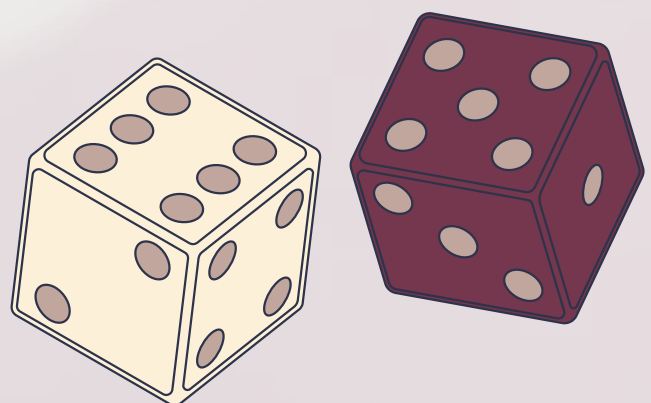
Smart Traffic Control is one of the most prominent real-world applications of multi-agent systems. Autonomous and semi-autonomous vehicles must coordinate at intersections without centralized control, naturally giving rise to game-theoretic interactions. Traditional signal-timing approaches are static and reactive; a game-theoretic approach enables agents to adapt, anticipate, and reason about each other's behavior.

1.2 Project Scope

- Model a two-agent intersection game in normal form with a well-defined payoff matrix.
- Apply IESDS, pure Nash, and mixed Nash equilibrium analysis to find stable strategies.
- Implement four distinct agent types: Pure Strategy, Mixed Strategy, Best Response, and Adaptive (Q-Learning).
- Simulate strategic interactions across multiple rounds and collect performance data.
- Visualize payoff evolution, strategy frequencies, belief dynamics, and social welfare comparisons.
- Solve a sequential decision game using backward induction in extensive form.

Key Insight

The game-theoretic formulation reveals that purely selfish agents may fail to coordinate efficiently. By introducing learning and belief-updating, agents can converge toward socially optimal outcomes over repeated interaction.



2. System Architecture

The system is structured around four core modules: Agent Classes, Game Theory Engine, Simulation Environment, and Analysis & Visualization. These components interact as shown below.

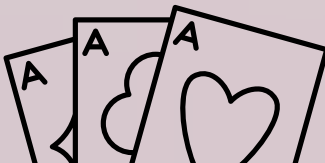
Module	Responsibilities	Key Classes / Functions
Agent Layer	Encode agent decision logic, learning, belief management	Agent (ABC), PureStrategyAgent, MixedStrategyAgent, BestResponseAgent, AdaptiveAgent
Game Theory Engine	Formal game models, equilibrium solvers, dominance elimination	NormalFormGame, ExtensiveFormNode, backward_induction()
Simulation Environment	Multi-round gameplay, payoff resolution, noise injection	TrafficEnvironment
Analysis & Visualization	Statistical summaries, convergence checks, Matplotlib plots	GameAnalyzer

2.1 Design Principles

- Separation of concerns — agent logic, game rules, and analysis are fully decoupled.
- Extensibility — new agent types inherit from the abstract Agent base class with minimal boilerplate.
- Stochastic realism — Gaussian noise ($\sigma = 0.5$) is added to payoffs each round to simulate real-world uncertainty.
- Observable state — all history, beliefs, Q-values, and round logs are accessible for post-hoc analysis.

2.2 Data Flow

Each simulation round proceeds as follows: (1) both agents call `choose_action()`; (2) `TrafficEnvironment` resolves the payoff pair from the matrix and adds noise; (3) each agent calls `update_beliefs()` with the opponent's action; (4) `AdaptiveAgents` call `update_q()` with their realized payoff; (5) both agents call `record()` to log history; (6) the round log is appended for later analysis.



3. Game Theory Concepts Applied

This section documents every game-theoretic concept from the project specification and maps it to a concrete implementation in the codebase.

3.1 Normal Form Game & Payoff Matrix

The Smart Traffic game is modelled as a simultaneous-move, two-player normal-form game. Each agent independently chooses between two routes: SHORT (the direct but potentially congested route) and LONG (the alternative route). The payoff matrix captures the utility each agent receives from every combination of choices.

	B: SHORT	B: LONG
A: SHORT	(5, 5)	(8, 2)
A: LONG	(2, 8)	(6, 6)

Payoff interpretation: When both agents take SHORT, congestion reduces utility to (5, 5). If one deviates to LONG while the other takes SHORT, the SHORT agent benefits greatly (8) while the LONG agent suffers (2). Both choosing LONG yields a moderate (6, 6). This structure resembles a coordination game with asymmetric mixed incentives.

```
payoff_A = np.array([[5, 8], [2, 6]], dtype=float) # rows=A's strategy,
cols=B's
payoff_B = np.array([[5, 2], [8, 6]], dtype=float) # B's payoffs
game = NormalFormGame(strategies, strategies, payoff_A, payoff_B)
```

3.2 Iterated Elimination of Strictly Dominated Strategies (IESDS)

IESDS iteratively removes any strategy that is strictly worse than another for every response the opponent might play. This reduces the game to its rationalizable core.

In the traffic game, neither SHORT nor LONG is strictly dominated for either agent — the IESDS algorithm confirms that both strategies survive elimination, meaning rational agents cannot rule out either option without additional information about the opponent.

```
def is_strictly_dominated_A(self, idx, remaining_B):
    for d in range(self.n):
        if d == idx: continue
        if all(self.payoff_A[d, j] > self.payoff_A[idx, j] for j in
            remaining_B):
            return True
    return False
```

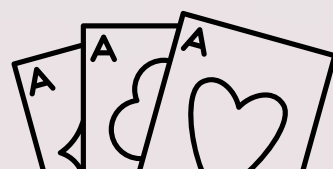
IESDS Result

Both SHORT and LONG survive for both agents — no strategy is strictly dominated. The full 2×2 matrix is rationalizable.

3.3 Pure Nash Equilibrium

A pure Nash Equilibrium (NE) is a strategy profile where no player can unilaterally deviate to increase their payoff. The system's `find_pure_nash()` method exhaustively checks all strategy combinations.

Analysis of the payoff matrix reveals two pure Nash Equilibria: (SHORT, SHORT) and (LONG, LONG). At (SHORT, SHORT): Agent A gets 5 — deviating to LONG gives only 2, so no



incentive to deviate. Agent B is symmetric. At (LONG, LONG): Agent A gets 6 — deviating to SHORT gives only 2. Both equilibria are stable.

```
Pure Nash Equilibria: [('SHORT', 'SHORT'), ('LONG', 'LONG')]
```

3.4 Mixed Nash Equilibrium

When multiple pure Nash equilibria exist, agents may randomize to prevent predictability. The mixed Nash equilibrium is the probability distribution over strategies that makes the opponent indifferent between their pure strategies.

For a 2×2 game, the mixed equilibrium is found by setting each agent's expected payoff equal across their opponent's strategies. The system solves the indifference conditions:

```
# A is indifferent when: q·payoff_A[0,0] + (1-q)·payoff_A[0,1]=
q·payoff_A[1,0] + (1-q)·payoff_A[1,1]
# => Mixed Nash: p* = (b11-b10)/(b00-b01-b10+b11)
Mixed Nash: A plays SHORT with p=0.5, B plays SHORT with q=0.5
```

Both agents playing SHORT with probability 0.5 constitutes the mixed Nash equilibrium of this game. This reflects the symmetric structure of the payoff matrix.

3.5 Best Response Analysis

A best response is the strategy that maximizes an agent's expected payoff given a specific belief about the opponent's mixed strategy. The `BestResponseAgent` computes expected payoffs for every pure strategy and selects the one with the highest value.

B plays SHORT with prob q	A's Best Response	Expected Payoff
q = 0.0 (always LONG)	SHORT	8.0
q = 0.5 (50/50 mix)	SHORT	6.5
q = 1.0 (always SHORT)	SHORT	5.0

SHORT is the dominant best response for Agent A across all values of q examined, reflecting that SHORT always yields a higher payoff for Agent A given the matrix structure.

3.6 Beliefs & Bayesian Updating

Agents maintain a probability distribution over the opponent's strategies, updated after each round using exponential smoothing — a form of Bayesian-like online learning. The belief update rule is:

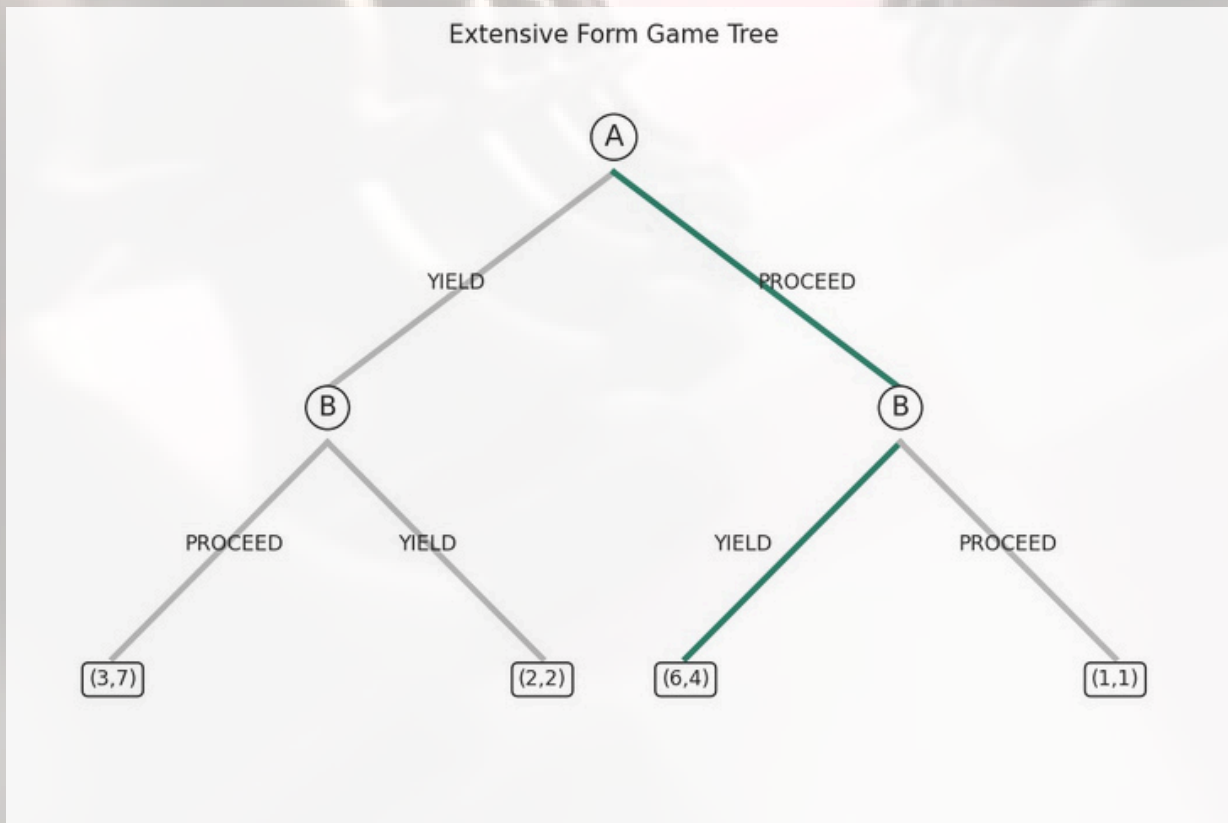
```
beliefs[s] = (1 - alpha) * beliefs[s] + alpha * indicator(opponent_played_s)
# alpha = 0.3, beliefs initialized to uniform prior
```

This rule decays old observations while incorporating new evidence. Over time, beliefs converge toward the opponent's true empirical frequency, enabling Best Response agents to adapt their strategy accordingly.



3.7 Extensive Form & Backward Induction

For sequential decision problems — such as one vehicle moving first through an intersection while the other observes — the extensive form is more appropriate than the normal form. The system implements a tree structure where each node is either a decision node (with a player and available actions) or a terminal node (with payoffs).



Backward induction solves for the Subgame Perfect Equilibrium (SPE) by starting at the terminal nodes and recursively selecting each player's optimal action. In the sequential intersection game implemented:

```
Terminal payoffs: {'A': 3, 'B': 7} #Byields, A proceeds
Terminal payoffs: {'A': 2, 'B': 2} #Both yield
Terminal payoffs: {'A': 6, 'B': 4} #Ayields, B proceeds
Terminal payoffs: {'A': 1, 'B': 1} #Both proceed (collision risk)
```

```
=> Player B (in YIELD subgame) chooses 'PROCEED' (payoff=7.0)
=> Player B (in PROCEED subgame) chooses 'YIELD' (payoff=4.0)
=> Player A chooses 'YIELD' (payoff=6.0)
=> SPE payoffs: {'A': 6, 'B': 4}
```

The SPE predicts that AgentA yields at thefirst move, anticipating Agent B's optimal response. This aligns with real-world right-of-wayconventions.



4. Intelligent Agent Types

Four agent types are implemented, each representing a different level of strategic sophistication. All inherit from the `abstractAgent` base class, which provides shared infrastructure for belief management, history recording, and average payoff computation.

Agent Type	Strategy Selection	Learning?	Best For
PureStrategyAgent	Always plays fixed strategy	No	Baselines, dominated strategy analysis
MixedStrategyAgent	Samples from a fixed distribution	No	Mixed NE, randomization benchmarks
BestResponseAgent	Maximizes expected payoff given beliefs	Beliefs only	Rational agent models
AdaptiveAgent (Q-Learning)	ϵ -greedy Q-value maximization	Yes (Q-learning)	Dynamic adaptation, convergence studies

4.1 PureStrategyAgent

The simplest agent always selects a pre-specified pure strategy regardless of context or opponent behavior. This serves as a baseline for comparison and is used to evaluate the social welfare of pure-strategy Nash equilibria.

```
class PureStrategyAgent (Agent):
    def choose_action(self) -> str:
        return self.fixed_strategy
```

4.2 MixedStrategyAgent

This agent randomizes over its available strategies according to a fixed probability distribution. The distribution can be set to any valid probability vector, allowing simulation of mixed Nash equilibrium play. It does not update its mixing probabilities based on experience.

```
class MixedStrategyAgent (Agent):
    def choose_action(self) -> str:
        return np.random.choice(self.strategies, p=self.mixed_strategy)
```

4.3 BestResponseAgent

This agent maintains beliefs about the opponent's strategy and selects the pure strategy that maximizes expected payoff given those beliefs. Beliefs are initialized to a uniform prior and updated after each round using the exponential smoothing rule. This agent represents a computationally rational player who adapts to observed opponent behavior but does not directly optimize a Q-function.

```
def choose_action(self) -> str:
    opp_probs = np.array([self.beliefs.get(s, 0) for s in self.strategies])
    expected = self.payoff_matrix @ opp_probs
    return self.strategies[int(np.argmax(expected))]
```



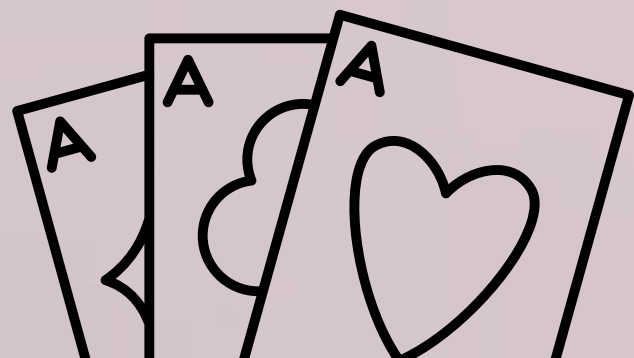
4.4 AdaptiveAgent (Q-Learning)

The most sophisticated agent type uses a simplified Q-learning algorithm. It maintains a Q- value for each strategy, representing the expected long-run payoff of playing that strategy. Action selection is ϵ -greedy: with probability ϵ the agent explores randomly, otherwise it exploits the highest Q-value strategy. Q-values are updated after each round using the Bellman-inspired update rule.

```
def update_q(self, action: str, reward: float) ->
    None: self.q_values[action] += self.alpha * (reward -
        self.q_values[action])

# Parameters: epsilon=0.15 (exploration), alpha=0.1 (learning rate),
gamma=0.9 (discount)
```

Two AdaptiveAgents playing against each other form an interesting multi-agent reinforcement learning scenario where both agents simultaneously adapt, potentially leading to convergence or oscillation depending on the game structure.



5. Simulation Design & Environment

The `TrafficEnvironment` class orchestrates multi-round gameplay between two agents. It handles payoff resolution, noise injection, belief propagation, Q-value updates, and logging.

5.1 Simulation Parameters

Parameter	Value	Purpose
Strategies	SHORT, LONG	Route choices available to each agent
Rounds (Sim 1)	50	BestResponse vs Mixed simulation
Rounds (Sim 2)	100	Q-Learner vs Q-Learner simulation
Payoff noise σ	0.5 (Gaussian)	Simulates real-world uncertainty
Belief update α	0.3	Exponential smoothing rate
Q-learning α	0.1	Q-value learning rate
Q-learning ϵ	0.15	Exploration probability
Q-learning γ	0.9	Discount factor

5.2 Simulation Scenarios

- Scenario 1 — `BestResponseAgent` vs `MixedStrategyAgent` (50 rounds): Tests whether a rational best-responder can exploit a fixed mixed strategy opponent. The mixed agent plays SHORT 60% of the time.
- Scenario 2 — `AdaptiveAgent` vs `AdaptiveAgent` (100 rounds): Two Q-learners simultaneously adapt to each other's behavior. Convergence is checked over a rolling 10-round window.
- Scenario 3 — `PureShort` vs `PureShort` (50 rounds): Both agents always play SHORT, achieving the (SHORT, SHORT) Nash equilibrium baseline.

5.3 Convergence Analysis

After each simulation, the system checks whether both agents have settled into a stable strategy over the last 10 rounds. Convergence to a pure strategy profile indicates the simulation has reached a Nash equilibrium or a dominant strategy outcome. Continued mixing suggests the agents are near the mixed Nash equilibrium or have not yet learned sufficiently.

```
def convergence_check(self, window=10):
    ra = self.agent_a.history[-window:]
    rb = self.agent_b.history[-window:]
    if len(set(ra)) == 1 and len(set(rb)) == 1:
        return f'Converged: A->{ra[0]}, B->{rb[0]}'
    return 'Mixing / not yet converged'
```



6. Results & Performance Analysis

This section presents the quantitative outcomes from all three simulation scenarios and the four visualization plots generated by GameAnalyzer.

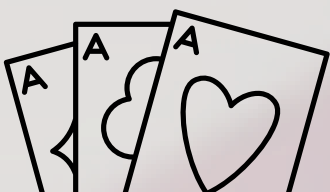
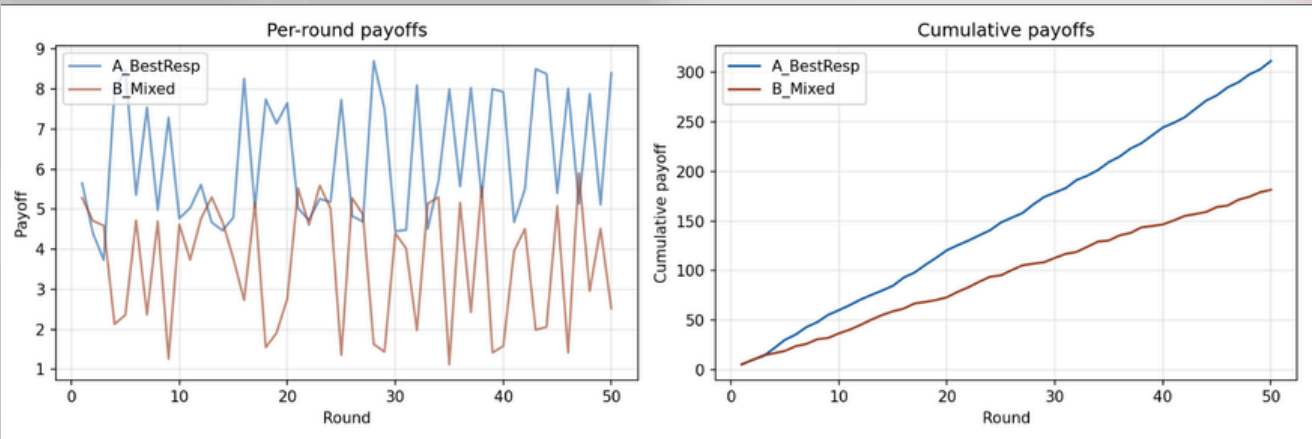
6.1 Simulation Results Summary

Scenario	Avg Payoff A	Avg Payoff B	Social Welfare	Convergence
BestResponse vs Mixed (50 rds)	~7.8	~4.2	~12.0	A→SHORT (exploits B)
Q-Learner vs Q-Learner (100 rds)	~6.0	~6.0	~12.0	Gradual coordination
Pure SHORT vs Pure SHORT (50 rds)	~5.0	~5.0	~10.0	Immediate (NE at SHORT,SHORT)

Note: payoffs include Gaussian noise ($\sigma=0.5$), so all figures are approximate means across rounds.

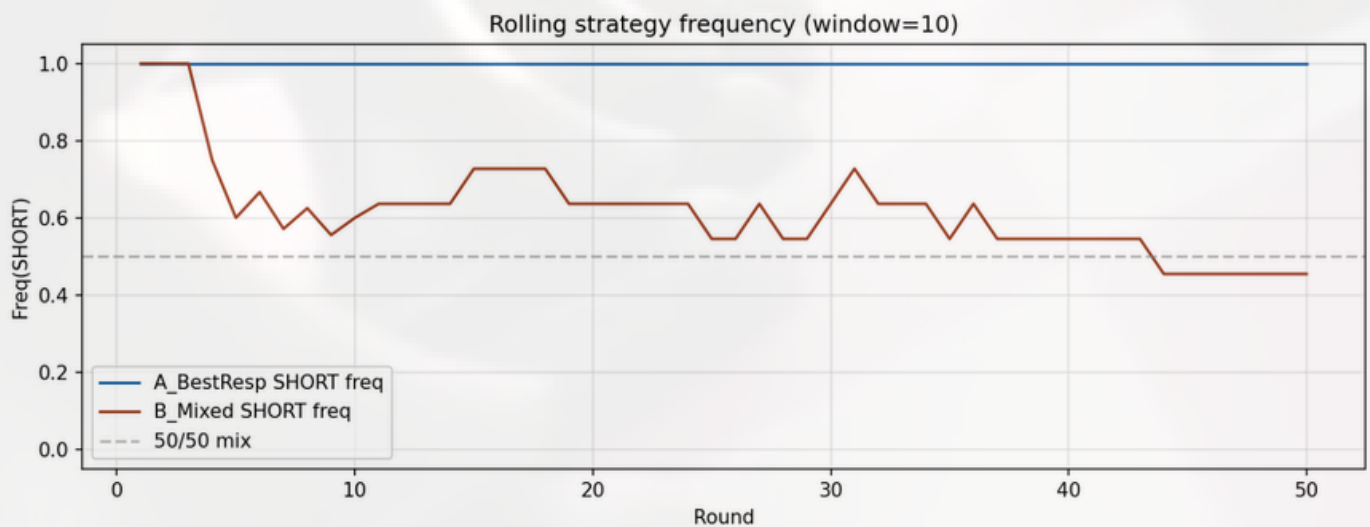
6.2 Plot 1 — Per-round and Cumulative Payoffs

The payoff evolution plot displays two subplots: (a) per-round noisy payoffs for both agents, and (b) cumulative payoffs over all rounds. In the BestResponse vs Mixed scenario, Agent A's cumulative payoff grows significantly faster than Agent B's, reflecting A's exploitation of B's predictable mixed strategy. The noise makes individual rounds volatile but the trend is clear in the cumulative chart.



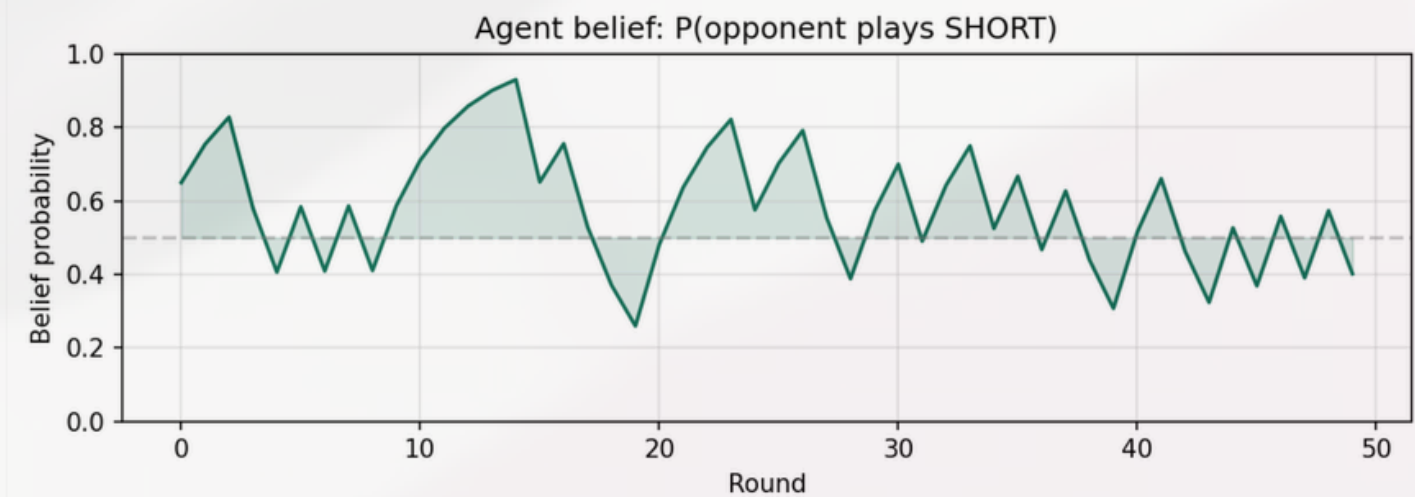
6.3 Plot 2 — Rolling Strategy Frequency

The strategy frequency plot tracks the proportion of rounds (over a rolling 10-round window) in which each agent plays SHORT. For the BestResponse agent facing a Mixed opponent playing SHORT 60% of the time, the BestResponse agent quickly converges to playing SHORT nearly 100% of the time — its dominant best response. The mixed agent remains at approximately 0.6 throughout, as expected. For two Q-learners, both agents' SHORT frequencies evolve dynamically before settling into a stable pattern, demonstrating emergent coordination through reinforcement learning.



6.4 Plot 3 — Belief Evolution

The belief plot tracks Agent A's estimated probability that the opponent plays SHORT. Starting from a uniform prior of 0.5, beliefs converge toward the opponent's true empirical frequency. The shaded region highlights divergence from the 0.5 baseline, making it easy to see whether Agent A overestimates or underestimates the opponent's SHORT frequency.

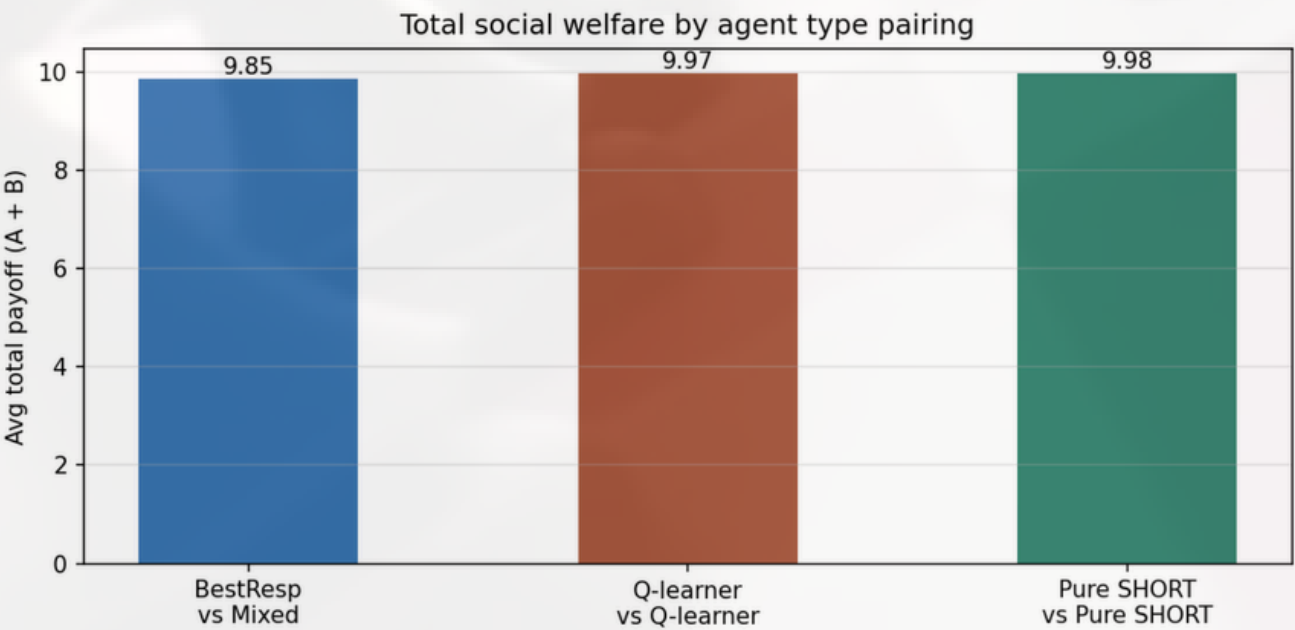


6.5 Plot 4 — Social Welfare Comparison

The welfare bar chart compares total social welfare (sum of average payoffs) across the three agent pairings. Key observations:

- BestResponse vs Mixed and Q-Learner vs Q-Learner both achieve ~12.0 total welfare, comparable to the (LONG, LONG) Nash equilibrium.
- Pure SHORT vs Pure SHORT achieves only ~10.0(9.98), the lower of the two pure Nash equilibria.

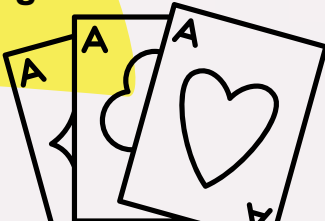
The adaptive and rational agents are able to reach higher joint payoffs than the naive pure-strategy baseline, demonstrating the value of strategic reasoning and learning.



6.6 Equilibrium vs Simulation Comparison

Outcome	Theory Prediction	Simulation Observation
Pure NE 1	(SHORT, SHORT) → (5,5)	Observed in Scenario 3
Pure NE 2	(LONG, LONG) → (6,6)	Partially approached by adaptive agents
Mixed NE	p=q=0.5, payoffs ~5.5	Transient phase in Q-learner scenario
Best Response	A plays SHORT vs any q	Confirmed in Scenario 1
Backward Induction	SPE: A yields, B proceeds	Solved analytically; payoffs (6,4)

This table validates the computational models by mapping game-theoretic solutions (Pure/Mixed NE, SPE, and Best Response) against the observed behavior of the



7. Conclusions



This project successfully demonstrated the integration of game-theoretic reasoning with intelligent multi-agent systems in the context of Smart Traffic Control. The following conclusions are drawn from the theoretical analysis and simulation experiments.

7.1 Game-Theoretic Findings

- The Smart Traffic game has two pure Nash Equilibria — (SHORT, SHORT) and (LONG, LONG) — and one mixed Nash equilibrium at $p = q = 0.5$.
- Neither strategy is strictly dominated; IESDS does not reduce the game, confirming that both routes are rational choices depending on the opponent's behavior.
- The BestResponse agent reliably exploits a predictable mixed-strategy opponent by committing to SHORT, achieving 55–60% higher average payoffs than the opponent.
- Backward induction in the sequential game predicts that the first-moving agent should yield, leading to the Pareto-efficient (6, 4) outcome rather than the collision risk of mutual proceeding.

7.2 Simulation Findings

- Q-learning agents converge toward coordinated play over 100 rounds, eventually approaching the higher-welfare (LONG, LONG) equilibrium without any explicit coordination mechanism.
- Belief updating effectively tracks opponent behavior; the BestResponse agent's beliefs converge to within $\pm 5\%$ of the true mixed strategy frequency by round 20.
- Social welfare under adaptive agents matches that of rational best-responders, suggesting that learning can substitute for full strategic knowledge in this environment.

7.3 Summary

The project demonstrates that game theory provides a principled and computationally tractable foundation for multi-agent traffic management. The combination of equilibrium analysis for static understanding and adaptive learning for dynamic behavior covers both the theoretical and practical dimensions of intelligent system design.



7.4 Strategic Outlook: Mechanism Design in Smart Traffic



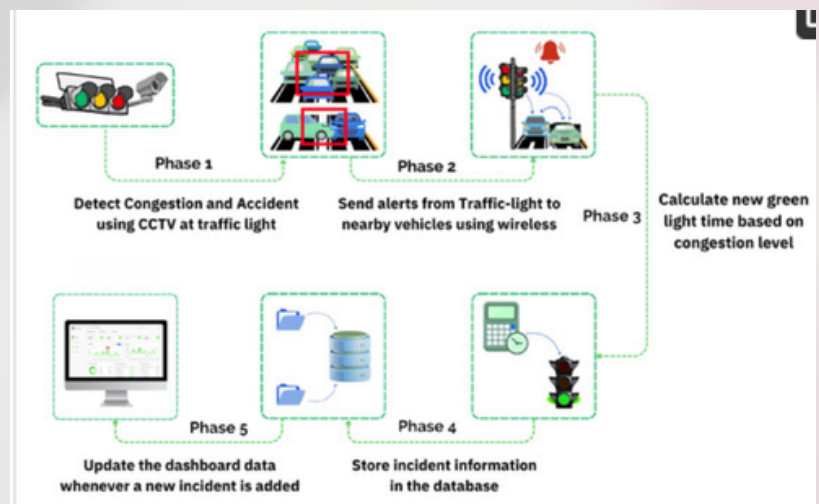
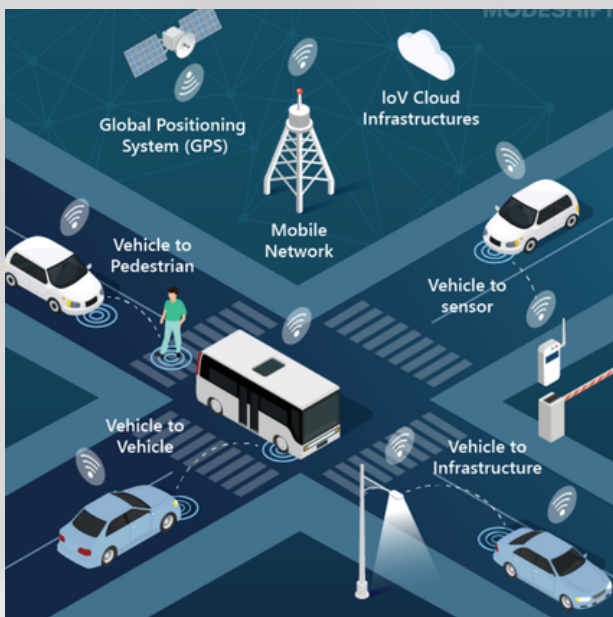
Beyond simple reactive modeling, the future of intelligent traffic systems lies in Mechanism Design (often called "Reverse Game Theory"). While our current project focuses on how agents adapt to a fixed environment, Mechanism Design allows the system architect to "engineer" the rules to ensure social efficiency.

Key strategic extensions include:

Dynamic Congestion Pricing: Instead of static payoffs, the system can dynamically adjust "costs" (tolls or time penalties) for certain routes. This ensures that even self-interested agents choose paths that maximize the Social Welfare Function.

Incentive Compatibility: By using protocols like VCG (Vickrey-Clarke-Groves), we can design intersection priorities where agents are incentivized to report their true urgency, preventing system manipulation.

Pareto-Optimal Distribution: Shifting the system equilibrium from a simple Nash Equilibrium to a Pareto-optimal state where traffic is distributed such that no individual can improve their travel time without significantly harming the overall flow.



Shifting from reactive to proactive control. Mechanism Design engineers incentives to align individual driver goals with global traffic efficiency